Exact Certification of the Kauers–Zeilberger Rook Recurrences in Dimensions Four through Twelve

Keio

2 November 2026

Abstract

We certify the original polynomial-coefficient recurrences conjectured by Kauers and Zeilberger for diagonal rook walks in every dimension $d = 4, \dots, 12$. This supplies a computer-assisted solution of the finite certification question stated as Problem 17 by Chen and Kauers. The argument represents the diagonal coefficients as powers of a Laguerre polynomial integrated against an exponential weight. Integration by parts reduces all required shifted integrals to a family of d moments. For each original recurrence, its residual is then checked to be the zero vector over $\mathbb{Q}[n]$, after clearing explicitly known denominators. The initial index $n = 0$ is checked separately from a finite combinatorial formula. Thus the conclusion holds for every nonnegative index, without an interpolation assumption, a modular reconstruction assumption, or an unproved minimal-order assertion. The integral reduction was established in corpus manuscript [067]; the contribution here is the direct exact certification of the published equations, including its five remaining dimensions 8 through 12. The single-file source contains the original data and a reproducible verifier. No claim of an unprecedented general principle or of universal recurrence minimality is made.

1 The original problem and the precise result

Let

$$R_d(x_1, \dots, x_d) = \frac{1}{1 - \sum_{i=1}^{d} x_i/(1 - x_i)}, \qquad a_n^{(d)} = [x_1^n \cdots x_d^n] R_d. \tag{1}$$

In particular $a_0^{(d)} = 1$. These integers count paths from $(0, \dots, 0)$ to $(n, \dots, n)$, each step increasing exactly one coordinate by an arbitrary positive integer.

Kauers and Zeilberger [1] published conjectured recurrences for these sequences through dimension twelve in 2010. Their computational challenge was to obtain rigorous certificates for those specific equations. Chen and Kauers [2, Problem 17] subsequently asked for proofs in dimensions $4, \dots, 12$. This is an existing numbered problem in symbolic computation, not an Erdős problem. It asks for the published equations themselves; merely proving the existence of some recurrence would not answer it.

Write the authors' equations in forward-shift form as

$$\sum_{j=0}^{d} K_{d,j}(n) a_{n+j}^{(d)} = 0, \qquad K_{d,j} \in \mathbb{Z}[n]. \tag{2}$$

Here the $K_{d,j}$ are the actual coefficients in the original public data [7], expanded without changing their normalization. Their complete integer coefficient lists are included in the source payload described in Section 6. The input hashes in that payload fix the precise source bytes; the coefficient hashes also fix the expanded polynomials. This is important because the largest list contains coefficients of more than two thousand bits.

Theorem 1.1 (Certification of the original equations)**.** *For each $d \in \{4, 5, \dots, 12\}$, the original Kauers–Zeilberger polynomials $K_{d,0}, \dots, K_{d,d}$ satisfy* (2) *for every integer $n \geq 0$.*

The proof consists of the analytic identities established below and finite exact polynomial calculations. Every coordinate of every cleared residual is verified to be identically zero, not only zero at some sample indices. Dimensions two and three are included as consistency checks and are not claimed as new results.

1.1 Relation to previous work

The three-dimensional recurrence was proved by Bostan, Chyzak, van Hoeij and Pech [3]. The general idea of combining an a priori bound with exact algebraic verification was already proposed in [1, Section 2]; reduction methods for rational integrals also have substantial prior literature, including [4, 5]. The supplied corpus manuscript [067] [6] develops the particular Laguerre moment reduction used here, proves an order-at-most-d theorem for every dimension, and reports certification through dimension seven. It explicitly leaves the original equations for dimensions eight through twelve uncertified. We credit that reduction and reproduce its proof so that the present certification is self-contained.

Our additional step is to construct the denominator-cleared reduction vectors directly over $\mathbb{Q}[n]$ and apply them to the original published coefficient lists. This eliminates the previous identification through guessed uniqueness and completes all the dimensions requested in Problem 17. The novelty asserted is this certification result and its reproducible implementation, not the underlying integration-by-parts principle. A targeted literature review on the date of this manuscript did not locate an earlier full certification of dimensions eight through twelve. That review is not a proof of absolute historical priority.

2 A one-variable integral for the diagonal

For $n \geq 1$ define

$$\ell_n(s) = \sum_{k=1}^{n} \binom{n-1}{k-1} \frac{s^k}{k!}. \tag{3}$$

This is $L_n^{(-1)}(-s)$ in the usual Laguerre notation, but no facts about special functions will be required beyond the identities proved here.

Lemma 2.1. *For every $d \geq 1$ and $n \geq 1$,*

$$a_n^{(d)} = \sum_{1 \leq k_1, \dots, k_d \leq n} \frac{(k_1 + \cdots + k_d)!}{k_1! \cdots k_d!} \prod_{h=1}^{d} \binom{n-1}{k_h - 1} \tag{4}$$

$$= \int_0^\infty e^{-s} \ell_n(s)^d \, ds. \tag{5}$$

Proof. Expand the geometric series in (1) and then use the multinomial theorem. For $n \geq 1$,

$$[x^n] \left(\frac{x}{1-x}\right)^k = \binom{n-1}{k-1} \quad (1 \leq k \leq n),$$

and the coefficient is zero for $k = 0$ and for $k > n$. Hence coefficient extraction leaves exactly the finite sum (4). Replacing $(k_1 + \cdots + k_d)!$ by $\int_0^\infty e^{-s} s^{k_1 + \cdots + k_d} \, ds$ and factoring the finite sum gives (5). There is no interchange of infinite sums with an integral. □

Lemma 2.2. *The following identities hold for $n \geq 1$:*

$$\ell_n(0) = 0, \qquad \ell_n'(0) = 1, \tag{6}$$
$$s\ell_n'' + s\ell_n' - n\ell_n = 0, \tag{7}$$
$$(n+1)\ell_{n+1} = (n+s)\ell_n + s\ell_n', \qquad \ell_{n+1}' = \ell_n + \ell_n'. \tag{8}$$

Proof. The first assertion follows from the first term of (3). Put $c_k = \binom{n-1}{k-1}/k!$, taking coefficients outside the allowed range to be zero. The coefficient of s^k in (7) is $(k+1)kc_{k+1} + (k-n)c_k = 0$ by the adjacent-binomial identity. For the first shift identity its coefficient of s^k is $(n+k)c_k + c_{k-1} = (n+1)\binom{n}{k-1}/k!$; the endpoint $k = n+1$ has coefficient $1/n!$ on both sides. The second shift identity is Pascal's identity, after comparing coefficients of s^{k-1}. □

3 The auxiliary moment reduction problem

Fix $d \geq 1$. For $0 \leq i \leq d$ and $k \geq i - d$ put

$$J_n(i,k) = \int_0^\infty e^{-s} s^k \ell_n(s)^{d-i} \ell_n'(s)^i \, ds. \tag{9}$$

The integrand is $O(s^{k+d-i})$ near zero and decays exponentially times a polynomial at infinity, so it is integrable. Negative powers of s in this notation cause no divergence in the stated range.

The auxiliary problem is to express every $J_n(i,k)$ with $k \geq 0$ using a fixed finite family of moments, with coefficients polynomial in n. Its solution supplies a certificate space for the original recurrence problem. The family is

$$J_n(0, 1-d),\ J_n(0, 2-d), \ldots, J_n(0,0). \tag{10}$$

We do not assume these d quantities are linearly independent.

Lemma 3.1 (Integration by parts)**.** *If $0 \leq i \leq d$ and $k \geq i - d + 1$, then*

$$(i+1)J_n(i,k) = kJ_n(i,k-1) + (d-i)J_n(i+1,k) + inJ_n(i-1,k-1), \tag{11}$$

where terms with zero coefficient are omitted.

Proof. Let $m_i = \ell_n^{d-i}(\ell_n')^i$. Equation (7) gives $m_i' = (d-i)m_{i+1} + in\, m_{i-1}/s - im_i$. The function $e^{-s}s^k m_i$ vanishes at both endpoints: at zero its order is at least $k + d - i \geq 1$. Every nonzero term in its derivative is integrable, since its lowest possible order at zero is $k + d - i - 1 \geq 0$. Integrating that derivative proves (11). □

Lemma 3.2 (Polynomial reduction)**.** *For $0 \leq i \leq d$, $k \geq 0$, there are effectively computable $\gamma_b^{(i,k)}(n) \in \mathbb{Q}[n]$, $1 - d \leq b \leq 0$, such that*

$$J_n(i,k) = \sum_{b=1-d}^{0} \gamma_b^{(i,k)}(n) J_n(0,b) \quad (n \geq 1). \tag{12}$$

All divisions in their construction are by nonzero integers depending on d and i, not by functions of n.

Proof. We give the elimination and its boundary cancellations explicitly. For $1 \le i \le d$, applying (11) with $i-1$ gives

$$(d-i+1)J_n(i,k) = iJ_n(i-1,k) - kJ_n(i-1,k-1) - (i-1)nJ_n(i-2,k-1). \tag{13}$$

It is valid whenever $k \ge i-d$. Terms of index $i-2$ are omitted when their coefficient is zero. Induction on i expresses this as

$$J_n(i,k) = \sum_{b=k-i}^{k} \lambda_b^{(i,k)}(n) J_n(0,b), \tag{14}$$

with polynomial coefficients. Only the first term on the right of (13) contributes to the largest index; only its second term contributes to the smallest. Consequently

$$\lambda_k^{(i,k)} = \binom{d}{i}^{-1}, \qquad \lambda_{k-i}^{(i,k)} = (-1)^i \binom{k}{i}\binom{d}{i}^{-1}. \tag{15}$$

For integer k, the binomial coefficient here means the falling factorial polynomial $k(k-1)\cdots(k-i+1)/i!$, so the formula also covers negative k.

Next take $i=d$ and $k \ge 1$ in (11), and substitute (14). Since $\lambda_k^{(d,k)} = 1$, this expresses $J_n(0,k)$ as

$$J_n(0,k) = \sum_{b=k-d-1}^{k-1} \mu_b^{(k)}(n) J_n(0,b), \qquad \mu_{k-d-1}^{(k)} = \frac{(-1)^d k}{d+1}\binom{k-1}{d}. \tag{16}$$

The last coefficient vanishes for $1 \le k \le d$. In this range the remaining indices are at least $k-d \ge 1-d$; for $k \ge d+1$ all indices are nonnegative and smaller than k. Induction on k now reduces all $J_n(0,k)$ with $k \ge 1-d$ into (10).

Finally, for $k \ge 0$ the smallest index in (14) is $k-i \ge -d$. The value $-d$ is possible only if $(i,k) = (d,0)$, where its coefficient is $(-1)^d\binom{0}{d} = 0$. Thus no extra moment is needed. The elimination divides only by $d-i+1$ and $d+1$, proving the last assertion as well. □

4 Shift transport and exact polynomial certificates

Define $P_0(n) = 1$ and $P_j(n) = \prod_{r=1}^{j}(n+r)$. Repeated use of (8) gives polynomial matrices

$$P_j(n)\begin{pmatrix} \ell_{n+j} \\ \ell'_{n+j} \end{pmatrix} = \begin{pmatrix} A_j & B_j \\ C_j & D_j \end{pmatrix}\begin{pmatrix} \ell_n \\ \ell'_n \end{pmatrix}. \tag{17}$$

All entries belong to $\mathbb{Z}[n,s]$. Start with $A_0 = D_0 = 1$ and $B_0 = C_0 = 0$; the complete recursion is

$$A_{j+1} = (n+j+s)A_j + sC_j, \qquad B_{j+1} = (n+j+s)B_j + sD_j, \tag{18}$$

$$C_{j+1} = (n+j+1)(A_j + C_j), \qquad D_{j+1} = (n+j+1)(B_j + D_j). \tag{19}$$

It follows by induction that each entry has total degree at most j in n, s.

Expanding the dth power of the first row and applying Lemma 3.2 defines polynomials

$$W_{j,b}(n) = \sum_{i=0}^{d}\binom{d}{i}\sum_{k\ge 0}[s^k](A_j^{d-i}B_j^i)\,\gamma_b^{(i,k)}(n), \qquad 1-d \le b \le 0. \tag{20}$$

The sums are finite, with $k \le dj$. Equations (5) and (17) give the exact identity

$$P_j(n)^d a^{(d)}_{n+j} = \sum_{b=1-d}^{0} W_{j,b}(n) J_n(0,b) \quad (n \ge 1). \tag{21}$$

These W vectors can be constructed without any sequence values and without guessing a recurrence.

Theorem 4.1 (Sufficient polynomial certificate)**.** *Let $K_0, \dots, K_d \in \mathbb{Q}[n]$. Construct W as in* (20) *and form the d polynomials*

$$T_b(n) = \sum_{j=0}^{d} K_j(n) \left(\frac{P_d(n)}{P_j(n)}\right)^d W_{j,b}(n), \qquad 1-d \le b \le 0. \tag{22}$$

If all T_b are the zero polynomial, then $\sum_{j=0}^{d} K_j(n) a^{(d)}_{n+j} = 0$ for every integer $n \ge 1$. If additionally $\sum_{j=0}^{d} K_j(0) a^{(d)}_j = 0$, the same recurrence holds for every integer $n \ge 0$.

Proof. Multiply the claimed recurrence by $P_d(n)^d$ and substitute (21). The result is

$$\sum_{b=1-d}^{0} T_b(n) J_n(0,b) = 0.$$

Every factor of $P_d(n)$ is a positive integer for $n \ge 1$, so it can be divided out. The additional hypothesis is exactly the missing index $n = 0$. No leading coefficient of the recurrence is divided out, so possible singular indices of that coefficient cause no exception. □

Remark 4.2. Vanishing of all the coordinate polynomials is a sufficient certificate; it is not asserted to be necessary for every identity among these integrals. Linear independence of the retained moments is unnecessary. Nor does the theorem assume that a guessed relation of a given order or degree is unique.

4.1 An optional finite-evaluation bound

The following bound from the same reduction explains why exact evaluation could also certify a candidate. The computations for the main theorem use direct symbolic vanishing instead.

Proposition 4.3. *Put $e = \lceil d/2 \rceil$. Then*

$$\deg_{\mathrm{n}} W_{j,b} \le \lfloor d/2 \rfloor + edj.$$

If $\max_j \deg K_j \le D$, every polynomial in (22) *has degree at most*

$$B_d(D) = D + \lfloor d/2 \rfloor + \lceil d/2 \rceil d^2. \tag{23}$$

In particular $B_d(D) + 1$ exact zero evaluations at distinct positive integers for each coordinate are sufficient.

Proof. In (13), only the term with index $i-2$ introduces a factor n. Therefore $\deg_{\mathrm{n}} \lambda^{(i,k)}_b \le \lfloor i/2 \rfloor$. Formula (16) has coefficient degrees at most $\max(\lfloor d/2 \rfloor, 1 + \lfloor (d-1)/2 \rfloor) = e$. Induction in k bounds the degree of a reduced coordinate of $J_n(0,k)$ by ek for $k \ge 0$. Lowering i first then gives $\deg_{\mathrm{n}} \gamma^{(i,k)}_b \le \lfloor i/2 \rfloor + ek$. Indeed, retained negative base indices have constant coordinates, and every intermediate index h satisfies $\max(0,h) \le k$.

Since A_j, B_j have total degree at most j, the coefficient $[s^k](A_j^{d-i} B_j^i)$ has n-degree at most $dj - k$. Each summand in (20) therefore has degree at most $dj - k + \lfloor i/2 \rfloor + ek \le \lfloor d/2 \rfloor + edj$. In (22), the extra factor $(P_d/P_j)^d$ has degree $d(d-j)$. Maximizing $D + d(d-j) + \lfloor d/2 \rfloor + edj$ over $0 \le j \le d$ gives (23). A polynomial with more roots than its degree is identically zero. □

5 Certification of the published coefficient lists

For $d \le 9$ we used the authors' Maple aggregate `ManuelRookRecurrences`; for $d = 10, 11, 12$ we used the original files `rooks10-rec.m`, `rooks11-rec.m`, and `rooks12-rec.m`. The aggregate contains the same displayed forward shifts $f[n+j]$. The parser allows only integer constants, the indeterminate n, parentheses, signs, addition, multiplication, and nonnegative integer powers. It checks the shift expression and that every shift occurs exactly once. It neither runs Maple nor executes the source as a program.

The coefficient lists are converted into exact integer polynomials. The moments and shifts are calculated over $\mathbb{Q}[n]$ using `python-flint` 0.9.0. For each b, equality of T_b with zero means equality of its entire normalized rational-polynomial object with zero; no floating-point operation or prime-modular inference is used. Exact polynomial quotients P_d/P_j are available because these products are explicitly nested.

For the remaining index $n = 0$, use $a_0^{(d)} = 1$ and let

$$G_n(s) = n!\ell_n(s) \in \mathbb{Z}[s], \qquad a_n^{(d)} = \frac{\sum_{h=0}^{dn} h![s^h]G_n(s)^d}{(n!)^d} \quad (n \ge 1).$$

The verifier computes this finite sum exactly for $1 \le n \le d+2$ and checks the original equation at zero, together with the additional consistency checks at $n = 1, 2$.

d	Order	Max. degree	Max. coefficient bits	Zero coordinates	$n = 0$
2	2	1	4	2	pass
3	3	4	18	3	pass
4	4	9	42	4	pass
5	5	18	104	5	pass
6	6	31	175	6	pass
7	7	50	317	7	pass
8	8	75	493	8	pass
9	9	108	784	9	pass
10	10	149	1027	10	pass
11	11	200	1549	11	pass
12	12	261	2022	12	pass

Table 1: Exact certification results. "Order" describes the supplied operator, not a proved minimal order. Bit length is that of the largest absolute expanded integer coefficient. Zero coordinates count polynomial identities in $\mathbb{Q}[n]$, not sampled sequence values.

Proof of Theorem 1.1. The original coefficient lists give the degrees and bit lengths in Table 1. The exact verifier reproduced in the source constructs (20) and obtains $T_b = 0$ for every one of the d coordinates in each dimension $4 \le d \le 12$. It also obtains zero for the independently computed $n = 0$ equation in every case. These are finite rational-arithmetic identities with the full inputs provided. Theorem 4.1 applies to each list and proves the original equation for every nonnegative integer index. □

For orientation, the two-dimensional input is

$$(n+2)a_{n+2}^{(2)} - (10n+14)a_{n+1}^{(2)} + 9na_n^{(2)} = 0.$$

Here $a_0, a_1, a_2 = 1, 2, 14$, and the zero-index check is $2 \cdot 14 - 14 \cdot 2 = 0$. The high-dimensional checks use precisely the same logical steps, with larger input polynomials.

5.1 Independent verification and computational scope

The cleared symbolic construction was cross-checked against a numerical exact-rational moment reduction at $n = 1, 3$ in every dimension two through twelve. A separate implementation of the literal λ and μ recursions agreed with the reduced vectors at $n = 1, 2, 7$ in those dimensions. These comparisons detect implementation errors but are not substituted for the polynomial identities used in the proof.

In addition, a second complete symbolic implementation was run for every dimension $8, 9, 10, 11, 12$. It constructs the literal λ coefficients and then the μ reduction, and transports shifts using the three-term identity

$$(n+1)\ell_{n+1} = (2n+s)\ell_n - (n-1)\ell_{n-1} \qquad (n \geq 2).$$

The initial forward shift is obtained from (8). All residual polynomials again vanished exactly, and separate zero-index checks passed. Its independently parsed input polynomials agreed with the supplied coefficient lists. The independent checker and its successful log are retained in the source payload.

The initial symbolic run took approximately 1.2, 3.0, 8.3, 17.3, and 43.8 seconds for dimensions eight through twelve, including the first cross-checks and the zero-index checks, on the research execution environment. These are observed timings, not hardware-independent complexity claims. In particular, construction of the certificate does not require the hundreds of large sequence values needed for guessing or long finite-evaluation runs.

6 Reproducibility and data identity

The accompanying single `.tex` file is both the complete typesetting source and a container for the proof inputs. After `\end{document}` it contains an ASCII-encoded compressed payload, bounded by the comment markers `ROOK-PAYLOAD-BEGIN` and `ROOK-PAYLOAD-END`. This payload includes the original authors' source bytes, the expanded coefficient lists, source URLs and SHA-256 digests, and the standalone exact verifier. The payload is ignored by TeX; compiling the paper requires no external data file or bibliography. The distributed PDF also carries this ZIP as an embedded attachment. The single-file TeX source is sufficient to recover it independently. Its contents are extracted for verification as follows.

```
# Save as extract_rook.py; run:
# python extract_rook.py PAPER.tex
from pathlib import Path
import base64, hashlib, io, sys, zipfile

lines = Path(sys.argv[1]).read_text(encoding="ascii").splitlines()
first = lines.index("% ROOK-PAYLOAD-BEGIN") + 1
last = lines.index("% ROOK-PAYLOAD-END")
encoded = "".join(line[2:] for line in lines[first:last])
payload = base64.b85decode(encoded)
expected = ("21bf5366e6df90c2d746e4cf22948353d"
            "7913a901577bd66222f134d09591b0d")
assert hashlib.sha256(payload).hexdigest() == expected
target = Path("rook_certificate")
target.mkdir(exist_ok=True)
with zipfile.ZipFile(io.BytesIO(payload)) as archive:
    for name in archive.namelist():
        assert Path(name).name == name
        (target / name).write_bytes(archive.read(name))
# Then run:
# python rook_certificate/verify_rook.py \
#   --data rook_certificate/certificate_data.json
```

After extraction, install `python-flint==0.9.0` in a Python environment and run the included verification command. No network access is needed during the certificate calculation. The verifier first confirms data identity, then performs the rational-polynomial checks and the zero-index checks. A successful run reports each dimension separately. The hashes are provenance and integrity checks; the mathematical proof is the zero-polynomial calculation together with Theorem 4.1.

6.1 What is and is not concluded

All nine dimensions asked for in Chen–Kauers Problem 17 are covered, with the original coefficients and the original sequence. No additional hypothesis on n, dimension, arithmetic, or asymptotics is imposed. This is an unconditional computer-assisted proof in the ordinary exact-arithmetic sense, not a proof checked by a formal proof assistant.

The result does not prove that order d is minimal in every dimension, does not settle a numbered Erdős problem, and does not establish that this work outranks unrelated results in the supplied corpus. The request for a historically unprecedented principle is also stronger than what is established here: the reduction has been explicitly credited to its existing source. The precisely achieved advance is the certification of the previously conjectured original equations throughout the specified finite range.

References

[1] M. Kauers and D. Zeilberger, *The computational challenge of enumerating high-dimensional rook walks*, Advances in Applied Mathematics **47** (2011), 813–819. First preprint: 21 November 2010. arXiv:1011.4671. DOI: 10.1016/j.aam.2011.03.004.

[2] S. Chen and M. Kauers, *Some open problems related to creative telescoping*, Journal of Systems Science and Complexity **30** (2017), 154–172; Problem 17. First preprint: 13 September 2016. arXiv:1609.03768. DOI: 10.1007/s11424-017-6202-9.

[3] A. Bostan, F. Chyzak, M. van Hoeij and L. Pech, *Explicit formula for the generating series of diagonal 3D rook paths*, Séminaire Lotharingien de Combinatoire **66** (2011), B66a. arXiv:1105.4456.

[4] A. Bostan, P. Lairez and B. Salvy, *Creative telescoping for rational functions using the Griffiths–Dwork method*, Proceedings of ISSAC 2013, 93–100. arXiv:1301.4313.

[5] P. Lairez, *Computing periods of rational integrals*, Mathematics of Computation **85** (2016), 1719–1752. arXiv:1404.5069. DOI: 10.1090/mcom/3054.

[6] Keio, *Problems from the literature settled by transferred methods*, manuscript [067], 23 October 2026, Section 1. Verifiable Mathematical Reasoning Corpus, repository snapshot `ecd87de3348dcb706a76bb12fde722d7a348a012`, retrieved 2 November 2026. Repository. This is prior corpus work, not an independent peer-review endorsement.

[7] M. Kauers and D. Zeilberger, original recurrence output accompanying [1], retrieved 2 November 2026. Maple aggregate through dimension nine; Kauers' original data directory (`rooks10-rec.m`, `rooks11-rec.m`, `rooks12-rec.m`). Complete source hashes and copies are retained in the source payload.

A Executable certificate construction

The following code is the standalone verification core included with the source payload. Polynomial arithmetic is exact. Its recursive moment elimination implements (13) and (16); the explicit assertion on the discarded $-d$ coordinate checks the boundary cancellation. The construction of the shifted vectors implements (18)–(20), and the final loop checks (22) and the separate zero-index identity.

```
#!/usr/bin/env python3
"""Exact verification of the ORIGINAL Kauers--Zeilberger rook recurrences.

Requires Python 3 and python-flint. No workspace imports or network access.
Usage: python transfer_rook_standalone.py --data transfer_rook_certificate_data.json
The data may instead be a ZIP containing certificate_data.json.

Checks source SHA256 hashes, strictly parses each original Maple recurrence,
compares all expanded integer coefficient vectors with the canonical data,
and checks exact polynomial identities in Q[n]. This is not sequence fitting.
"""
from pathlib import Path
import argparse
import ast
import hashlib
import json
import re
import time
import zipfile
from math import comb
from flint import fmpq as Fraction, fmpq_poly, fmpz_poly

def require(condition, message):
    if not condition:
        raise ArithmeticError(message)

def coefficient_bytes(coefficients):
    """Canonical bytes: json.dumps(list-of-lists), ASCII, no terminal newline."""
    return json.dumps(coefficients, ensure_ascii=True).encode('ascii')

def digest(blob):
    return hashlib.sha256(blob).hexdigest()

def safe_polynomial(expression, variable):
    """Interpret integer polynomial syntax; never execute source code."""
    def evaluate(node):
        if isinstance(node, ast.Constant) and type(node.value) is int:
            return node.value
        if isinstance(node, ast.Name) and node.id == 'n':
            return variable
        if isinstance(node, ast.UnaryOp) and isinstance(node.op, (ast.USub, ast.UAdd)):
            value = evaluate(node.operand)
            return -value if isinstance(node.op, ast.USub) else value
        if isinstance(node, ast.BinOp):
            left, right = evaluate(node.left), evaluate(node.right)
            if isinstance(node.op, ast.Add):
                return left + right
            if isinstance(node.op, ast.Sub):
                return left - right
            if isinstance(node.op, ast.Mult):
                return left * right
            if isinstance(node.op, ast.Pow) and type(right) is int and 0 <= right <= 1000:
                return left ** right
        raise ValueError('Unsupported polynomial syntax: ' + ast.dump(node))
    clean = expression.replace('^', '**').replace('\n', ' ').strip()
    return evaluate(ast.parse(clean, mode='eval').body)

def parse_original(d, source_text):
    if d <= 9:
        marker = f'RRope[{d}]:='
        require(source_text.count(marker) == 1, f'd={d}: ambiguous source marker')
        body = source_text.split(marker, 1)[1].split(':', 1)[0]
```

```
    else:
        body = source_text.strip().rstrip(':;')
    body = body.replace('\\\n', '')
    variable = fmpz_poly([0, 1])
    coefficients = [None] * (d + 1)
    end = 0
    for match in re.finditer(r'f\[([^]]+)\]', body):
        index = fmpz_poly(safe_polynomial(match.group(1), variable))
        j = int(index(0))
        require(index == fmpz_poly([j, 1]), f'd={d}: index is not exactly n+j')
        require(0 <= j <= d, f'd={d}: invalid shift {j}')
        require(coefficients[j] is None, f'd={d}: duplicate shift {j}')
        expression = body[end:match.start()].strip()
        require(expression.endswith('*'), f'd={d}: missing coefficient product')
        coefficients[j] = fmpz_poly(safe_polynomial(expression[:-1], variable))
        end = match.end()
    require(not body[end:].strip(), f'd={d}: unparsed trailing source')
    require(all(p is not None for p in coefficients), f'd={d}: missing shift')
    require(coefficients[d] != 0, f'd={d}: leading coefficient is zero')
    return coefficients

def rook_initial_values(d, maximum):
    """Direct composition formula using integer polynomial arithmetic.

    G_n(s)=sum_{k=1}^n binom(n-1,k-1)*(n!/k!)*s^k.
    A_d(n)=sum_K [s^K]G_n(s)^d*K!/(n!)^d; A_d(0)=1.
    """
    factorials = [1]
    for k in range(1, d * maximum + 1):
        factorials.append(k * factorials[-1])
    values = [1]
    for n in range(1, maximum + 1):
        g = fmpz_poly([0] + [comb(n - 1, k - 1) *
                             (factorials[n] // factorials[k])
                             for k in range(1, n + 1)])
        numerator = sum(int(c) * factorials[k] for k, c in enumerate(g ** d))
        denominator = factorials[n] ** d
        require(numerator % denominator == 0, 'Nonintegral direct rook number')
        values.append(numerator // denominator)
    return values

def cleared_vectors(d, jmax=None):
    """Return W[j][b] with P_j(n)^d A_d(n+j)=sum_b W[j][b] J_n(0,b).

    P_j=product_{h=1}^j(n+h), b runs from 1-d through 0.
    All computations are exact in Q[n]; only constant divisions occur.
    """
    n0 = fmpq_poly([0,1])
    if jmax is None:
        jmax = d
    memo = {}
    cdiag = [Fraction(1)]
    for i in range(d):
        cdiag.append(cdiag[-1]*Fraction(1+i, d-i))
    def add(u, v, c=1):
        w = dict(u)
        for key, val in v.items():
            w[key] = w.get(key, 0) + c*val
        return {kk: vv for kk, vv in w.items() if vv != 0}
    def scale(u, c):
        return {kk: c*vv for kk, vv in u.items() if c*vv != 0}
    def low(i, k):
        key = ('low', i, k)
        if key in memo:
            return memo[key]
        if i == 0:
            res = {}
        else:
            part = scale(low(i-1, k), i)
            part = add(part, scale(J(i-1, k-1), -k))
            if i >= 2:
                part = add(part, scale(J(i-2, k-1), -(i-1)*n0))
```

```
            res = scale(part, Fraction(1, d-i+1))
        memo[key] = res
        return res
    def J0(k):
        key = ('J0', k)
        if key in memo:
            return memo[key]
        if k <= 0:
            require(k >= -d, 'Invalid base moment')
            res = {k: Fraction(1)}
        else:
            rhs = add(scale(J(d, k-1), k), scale(J(d-1, k-1), d*n0))
            rhs = scale(rhs, Fraction(1, 1+d))
            res = scale(add(rhs, low(d, k), -1), 1/cdiag[d])
        memo[key] = res
        return res
    def J(i, k):
        key = ('J', i, k)
        if key in memo:
            return memo[key]
        require(k >= i - d, 'Invalid reduced moment')
        if i == 0:
            res = J0(k)
        else:
            res = add(scale(J0(k), cdiag[i]), low(i, k))
        memo[key] = res
        return res
    # Cleared shift polynomials A_j,B_j,C_j,D_j, represented in powers of s.
    def padd(p, q):
        r = [Fraction(0)]*max(len(p), len(q))
        for i, x in enumerate(p): r[i] += x
        for i, x in enumerate(q): r[i] += x
        return r
    def pmul(p, q):
        r = [Fraction(0)]*(len(p)+len(q)-1)
        for i, x in enumerate(p):
            if x:
                for j, y in enumerate(q):
                    if y:
                        r[i+j] += x*y
        return r
    vecs = []
    a, b, ap, bp = [Fraction(1)], [Fraction(0)], [Fraction(0)], [Fraction(1)]
    for j in range(jmax+1):
        if j > 0:
            m = n0 + j - 1
            # P_j*l_{n+j}=a*l_n+b*l_n'; P_j*l_{n+j}'=ap*l_n+bp*l_n'.
            na = [x for x in padd(pmul([m, Fraction(1)], a), pmul([Fraction(0), Fraction(1)], ap))]
            nb = [x for x in padd(pmul([m, Fraction(1)], b), pmul([Fraction(0), Fraction(1)], bp))]
            nap = [(m+1)*x for x in padd(a, ap)]
            nbp = [(m+1)*x for x in padd(b, bp)]
            a, b, ap, bp = na, nb, nap, nbp
        tot = {}
        for i in range(d+1):
            # binom(d,i) a^{d-i} b^i
            P = [Fraction(comb(d, i))]
            for _ in range(d-i):
                P = pmul(P, a)
            for _ in range(i):
                P = pmul(P, b)
            for k, c in enumerate(P):
                if c:
                    for key, val in J(i, k).items():
                        tot[key] = tot.get(key, 0) + c*val
        require(tot.get(-d, 0) == 0, f'd={d}, j={j}: forbidden boundary moment')
        vecs.append([tot.get(k, Fraction(0)) for k in range(1-d, 1)])
    return vecs

def load_data(path):
    if zipfile.is_zipfile(path):
        with zipfile.ZipFile(path) as archive:
            return json.loads(archive.read('certificate_data.json'))
```

```
    return json.loads(path.read_bytes())

def check_dimension(d, data):
    start = time.perf_counter()
    record = data['recurrences'][str(d)]
    source = data['sources'][record['source']]
    source_bytes = source['text'].encode('ascii')
    require(digest(source_bytes) == source['sha256'], f'd={d}: source digest mismatch')
    K = parse_original(d, source['text'])
    coefficients = [[int(c) for c in polynomial] for polynomial in K]
    require(coefficients == record['coefficients'], f'd={d}: original coefficients mismatch')
    require(digest(coefficient_bytes(coefficients)) == record['coefficient_sha256'],
            f'd={d}: coefficient digest mismatch')
    W = cleared_vectors(d)
    P = [fmpq_poly([1])]
    for j in range(1, d + 1):
        P.append(P[-1] * fmpq_poly([j, 1]))
    for b in range(d):
        residual = fmpq_poly([0])
        for j in range(d + 1):
            quotient, remainder = divmod(P[d], P[j])
            require(remainder == 0, 'Unexpected denominator remainder')
            residual += fmpq_poly(K[j]) * quotient ** d * W[j][b]
        require(residual == 0, f'd={d}: nonzero residual at b={b + 1 - d}')
    # The integral/moment proof starts at n=1; n=0 is a separate exact check.
    # n=1,2 are additional independent finite-sum sanity checks.
    values = rook_initial_values(d, d + 2)
    for n in (0, 1, 2):
        require(sum(K[j](n) * values[n+j] for j in range(d + 1)) == 0,
                f'd={d}: failed direct recurrence check at n={n}')
    result = {
        'dimension': d,
        'order': d,
        'coefficient_degree': max(p.degree() for p in K),
        'max_coefficient_bits': max(abs(c).bit_length() for p in coefficients for c in p),
        'coefficient_sha256': record['coefficient_sha256'],
        'source_sha256': source['sha256'],
        'symbolic_residual_zero_count': d,
        'direct_initial_checks': [0, 1, 2],
        'seconds': time.perf_counter() - start,
    }
    print(json.dumps(result), flush=True)
    return result

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--data', type=Path,
                        default=Path(__file__).with_name('transfer_rook_certificate_data.json'))
    parser.add_argument('--dimensions', type=int, nargs='+', default=list(range(2, 13)))
    parser.add_argument('--report', type=Path)
    parser.add_argument('--extract-sources', type=Path)
    args = parser.parse_args()
    require(all(2 <= d <= 12 for d in args.dimensions), 'Dimensions must be between 2 and 12')
    data = load_data(args.data)
    require(data['format'] == 'rook-exact-certificate-v1', 'Unrecognized data format')
    if args.extract_sources:
        args.extract_sources.mkdir(parents=True, exist_ok=True)
        for filename, source in data['sources'].items():
            require(Path(filename).name == filename, 'Unsafe embedded filename')
            args.extract_sources.joinpath(filename).write_bytes(source['text'].encode('ascii'))
    report = [check_dimension(d, data) for d in args.dimensions]
    if args.report:
        args.report.write_text(json.dumps(report, indent=2) + '\n')
    print('PASS: exact original-coefficient recurrence identities for dimensions ' +
          ', '.join(map(str, args.dimensions)), flush=True)

if __name__ == '__main__':
    main()
```